

Lab: Discovering IBM Bob Premium Package for Z

Analysis and Documentation of a CICS Mainframe Application

Estimated Duration: 2-3 hours

Level: Intermediate to Advanced

Prerequisites: Basic knowledge of COBOL and CICS



Table of Contents

1. [Exercise 0: Lab Preparation](#)
 2. [Exercise 1: Workspace Initialization and Analysis](#)
 3. [Exercise 2: Define Naming and Organization Rules](#)
 4. [Exercise 3: Metadata Scan and Data Dictionary](#)
 5. [Exercise 4: Application Inventory Generation](#)
 6. [Exercise 5: Coding Standards Skill Creation](#)
 7. [Exercise 6: Architecture Diagram Creation](#)
 8. [Exercise 7: BANKDATA Program Documentation](#)
 9. [Exercise 8: Business Rules Extraction and Code Modification](#)
 10. [Exercise 9: Variable and Data Usage Analysis](#)
 11. [Exercise 10: Change Impact Analysis](#)
 12. [Exercise 11: User Journey Documentation](#)
 13. [Exercise 12: Email Search Implementation](#)
 14. [Summary and Measurable Gains](#)
 15. [Conclusion](#)
 16. [Appendix: Useful Prompt Reference](#)
 17. [Next Steps: Beyond the Lab](#)
-

Exercise 0: Lab Preparation

Objective

Retrieve the CBSA application source code from GitHub and prepare the workspace for the lab.

Prerequisites

- Have IBM Bob Version 2 installed on a workstation (MacOS, Linux, or Windows)
- Have the **IBM Bob Premium Package for Z** extension installed (version 3.0.0 or higher)
 - The following extensions should be automatically installed with it:
 - Zowe Explorer 3.5.0 (or higher)
 - IBM Z Open Editor 6.6.0 (or higher)
 - Mermaid (latest version)

Bob Mode to Use

Mode:  **Agent**

Agent mode allows executing system commands and manipulating files.

Context

Before starting the analysis, you need to retrieve the CBSA application source code from the official GitHub repository. We will use Bob to automate this preparation. First, create a CBSA directory in your home directory. Open Bob IDE, click on the **File>Open Folder** menu and choose the ~/CBSA directory.

Bob Prompt

```
Retrieve the sub-directory named "src/base" in the CBSA directory from the GitHub repository https://github.com/ovallo/Bob4z-a-thon.git and place it at src/base in the current workspace folder. Then remove any temporary working directory you would have created.
```

Expected Outcome

Bob retrieves the `src/base` sub-directory from the GitHub repository, places it in the workspace, and cleans up any temporary directories it created.

 **Observe how Bob approaches this:** the method may vary depending on your environment. If `git` is installed, Bob will likely clone the repository and extract the relevant sub-directory. If `git` is not available, Bob may find an alternative — for example, downloading the repository as a ZIP archive from GitHub and expanding it. It is interesting to observe which strategy Bob picks.

Key Takeaways

- **Adaptability:** Bob selects the best available approach to accomplish the task (git clone, ZIP download, etc.)
- **Automation:** Bob can execute system commands and handle file operations end-to-end
- **Workspace preparation:** Bob organizes the retrieved files cleanly in the workspace
- **Verification:** Bob confirms everything is in place before handing back control
- **Time savings:** A few minutes instead of 15–20 minutes manually

You're Ready!

Once the workspace is prepared with Bob, you can click on “New Task” and start Exercise 1.

Exercise 1: Workspace Initialization and Analysis

Objective

Initialize the workspace and create the `AGENTS.md` file that will serve as a guide for Bob and developers.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

Context

You have a CBSA project that you are discovering. Before doing any analysis, Bob needs to understand the workspace structure — which languages are used, where the files are, and what coding standards or documentation already exist. The `/init` command takes care of all of this in one step.

Bob Prompt

 In Bob's prompt area, type `/` to open the slash command menu, then select `/init` or finish typing it and press Enter.

```
/init
```

Note: `/init` is a special IBM Bob Premium Package for Z command that triggers a complete workspace analysis and creates or updates `AGENTS.md` with everything it discovers.

What Bob Does Automatically

Bob will:

- Analyze the COBOL workspace (structure, languages, files)
- Detect present mainframe languages
- Check for data dictionary existence (bobz/DD.json)
- Search for coding standards
- Locate technical documentation
- Map COBOL programs to their documentation
- Display detected configuration for confirmation
- Update AGENTS.md with non-obvious information

Expected Outcome

Bob creates or updates `AGENTS.md` at the workspace root with a summary of what it discovered: detected languages, file locations, existing documentation, and any coding standards found. You can open the file to verify its contents before moving on to the next exercise.

Exercise 2: Define Naming and Organization Rules

Objective

Before starting your project analysis, establish the rules and conventions to apply. For example, clear conventions for organizing and naming resources generated by Bob to maintain a structured and consistent workspace. And specify that documentation will be written in English.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

Bob Prompt

```
Create the following Bob rules in .bob/rules/cbsa-workspace-rules.md:
- Documents must be stored in the docs/ directory. Documents specific to a program should be stored in a sub-directory of docs/ using the program name. For example, documents about BNKMENU should be stored in docs/BNKMENU/ directory.
- Tools must be stored in the tools/ directory.
- Schemas, drawings and graphs must be stored in graph/ directory.
```

Document, schemas, drawings and graphs file names must follow these conventions:

```
-- Prefix: program name (e.g., BNKMENU) if the document concerns a specific program, or CBSA for the application, or GLOBAL for cross-cutting documents
```

```
-- Document type: analysis, archi, docu, inv, plan, spec
```

```
-- Format: [PREFIX]-[TYPE]-[description].md
```

Also add the following note to AGENTS.md: "Always respect the rules defined in the .bob/rules/ directory."

Expected Outcome

Bob creates `.bob/rules/cbsa-workspace-rules.md` with the conventions you specified, and adds a note to `AGENTS.md` referencing that file. From this point on, Bob will automatically apply these rules to all generated files and directories throughout the lab.

Key Takeaways

- `.bob/rules/` is a directory — each `.md` file placed inside it is automatically picked up by Bob and applied to all interactions
- `AGENTS.md` serves a different purpose: it holds descriptive project context (structure, technologies, patterns) generated by `/init`
- **Bob adapts to your standards:** whatever naming and organizational conventions your team already uses, Bob can be instructed to follow them

 **Be precise with Bob.** Bob takes your instructions literally — if you ask it to create a file “in `.bob/rules`” without specifying a filename, it may create a file *called* `rules` with no extension rather than a file inside a `rules` directory. The more specific your prompt (exact paths, filenames, and formats), the more predictable the result. This applies especially when enforcing standards: if your team has existing conventions, spell them out explicitly rather than assuming Bob will infer them.

Exercise 3: Metadata Scan and Data Dictionary

Objective

Build the local metadata database by scanning the COBOL programs, then generate the data dictionary. These two artefacts are the foundation for all subsequent analyses — they allow Bob to give accurate, context-aware answers about the codebase.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

Context

Now that the workspace is initialized and naming rules are in place, Bob needs deeper knowledge of the code itself. The metadata scan parses all COBOL programs and stores structural information in a local database. The data dictionary then builds on that to document the key business variables used across the application.

Bob Prompt

```
Create the local metadata database with the scan_program tool.
```

 **Language Selection:** If prompted for which language you want to build the metadata repository, select **COBOL**, as this lab focuses on the COBOL code.

 **zapp.yaml Warning:** A warning may pop up about the `zapp.yaml` file that is being created. You can safely disregard it for the purpose of this lab.

Note: The metadata database is an IBM Bob Premium Package for Z feature that stores structural information about your COBOL programs, enabling more accurate and context-aware analysis throughout the lab.

Generate the Data Dictionary

To initiate the workflow: 1. Click the **Start Workflow** icon (Play button in Bob). 2. Select **“Generate data dictionary”** and click **Start**. 3. When Bob asks you to select the program from which the data dictionary will be built, choose `BANKDATA.cb1` (located in `src/base/cobol_src/BANKDATA.cb1`; note that you can only select one program). `BANKDATA.cb1` is the main data initialization program for the CBSA application, making it the ideal starting point for extracting core business variables. 4. Bob will present the generated data dictionary entries for review. While this interactive review allows refining business terms in real-world scenarios, for the purpose of this lab, simply click **“I’m done editing”** to proceed.

What Bob Does Automatically

Bob will use specific skills and tools from the IBM Bob Premium Package for Z. It will: - Verify the existence of a metadata database by reading `.bobz/local-settings.json` - Scan COBOL programs if necessary - Extract variables from `BANKDATA.cb1`

Expected Outcome

Bob generates `bobz/DD.json` containing documented business variables from `BANKDATA.cb1`. `AGENTS.md` is updated with the location of the data dictionary.

Key Takeaways

- **Foundational metadata & Data Dictionary:** Creating the local metadata database (`.bobz/local-settings.json`) and data dictionary (`bobz/DD.json`) establishes the structural and semantic foundation required for accurate downstream analysis.

- **Semantic understanding:** The data dictionary bridges raw COBOL variables to business terms, allowing Bob to reason about application logic and business rules rather than just syntactic structure.
 - **Underlying Z Tooling:** Workflows coordinate specialized Z tools (like `scan_program` , `get_variables` , and `edit_data_dictionary`) under the hood to perform complex, multi-step analysis automatically.
 - **Time savings:** Instead of spending 2-3 days manually exploring the code and cross-referencing variables, Bob analyzes everything and builds the dictionary in a few minutes.
-

Exercise 4: Application Inventory Generation

Objective

Generate a complete application inventory with all programs, copybooks, BMS screens, Db2 tables, and VSAM files.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode automatically analyzes mainframe application structure and generates detailed inventories.

Context

Now that the workspace is initialized, you need a detailed inventory to:

- Know all application components
- Understand program categories
- Identify dependencies
- Have an overview of the architecture

Bob Prompt

Generate a complete inventory of the CBSA application, with for each program, their type, role, and dependencies (used copybooks, BMS screens, DB2 tables and files used - with access mode -, queues, and called programs).

Expected Outcome

Bob queries the metadata database using `execute_sql_query` and generates a comprehensive inventory document in the `docs/` directory, respecting the naming conventions defined in Exercise 2 (for example, `docs/CBSA-inv-application-inventory.md`).

The generated document includes:

- **Executive summary:** Key statistics and high-level breakdown of application components
- **Component inventory:** Detailed listing of COBOL programs, their designated roles, and types (batch vs. online)
- **Dependency mapping:** Associated copybooks, BMS screens, Db2 tables (with access modes), VSAM files, queues, and called programs - ... and more

Key Takeaways

- **Rules applied automatically:** The generated inventory is automatically named and filed in `docs/` according to the conventions defined in Exercise 2, without requiring reminders in the prompt.
- **Structured database queries over text search:** By using `execute_sql_query` behind the scenes, Bob retrieves precise program types, file access modes, and dependencies that simple text searches often miss.

 **Putting the pieces together:** This exercise shows how the earlier setup steps directly connect — Bob leverages the local metadata database generated in Exercise 3 to extract comprehensive relationships, and structures the output according to the workspace rules from Exercise 2.

Exercise 5: Coding Standards Skill Creation

Objective

Discover and document current patterns and standards to apply them to new code (or assert them on existing code).

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

Context

Now that the workspace is initialized and you have a detailed inventory, you need to discover and document the existing codebase conventions — such as coding practices, naming patterns, and error handling standards — so they can be enforced consistently.

Bob Prompt

 In Bob's prompt area, type `/` to open the slash command menu, then select `/z-coding-standards-skill-builder` or finish typing it and press Enter.

```
/z-coding-standards-skill-builder
```

 **Directory selection:** When Bob asks which directory to scan for coding standards analysis, select `src/base/` (Programs, Copybooks and BMS-Maps).

 **ZCodeScan:** When Bob asks whether to include ZCodeScan validation instructions in the new skill, select or reply **Yes**. This embeds automated code-quality checks directly into the skill so Bob can run ZCodeScan validations as part of every standards review.

What Bob Does Automatically

Bob will scan, analyze, and document application components. It will use the IBM Bob Premium Package for Z command `/z-coding-standards-skill-builder` to analyze the code, variable and paragraph names, and error handling.

At the end of Bob's output, you will find examples on how to activate this new skill.

Expected Outcome

Bob generates a tailored coding standards skill packaged at `.bob/skills/cbsa-coding-standards/SKILL.md`, along with detailed reference files in the `.bob/skills/cbsa-coding-standards/references/` sub-directory.

The generated skill encapsulates: - **Application patterns:** Program structure, division layouts, and standard linkage conventions - **Naming standards:** Prefixing and formatting rules for variables, paragraphs, and sections - **Error handling practices:** Standard CICS RESP/RESP2 evaluation and ABNDPROC routing routines - **Verification checklist:** Concrete criteria used to audit existing code or validate newly generated snippets - ... and more

Testing the New Skill (in a new task)

 **Reload Workspace:** To ensure the new skill is discovered, reload your workspace. Close the folder (**File > Close Folder**) then reopen it (**File > Open Folder**).

 **Switch Mode:** Workspace reloading resets the active mode. Remember to switch back to **Z Code** mode in Bob before running the prompt below.

```
verify @src/base/cobol_src/BNK1CAC.cbl is respecting cbsa coding standards
```

For specific lines of code only: - Either select a code block, then **Right Click > IBM Bob > Improve code** - Or use the prompt:

```
Improve the following code from src/base/cobol_src/BNK1CAC.cbl:239-254
```

Key Takeaways

- **Codifying implicit conventions:** The `/z-coding-standards-skill-builder` command extracts unwritten codebase rules (naming patterns, paragraph structures, error handling) into a structured skill (`SKILL.md`).
 - **Reusable quality gate:** Once generated, the skill can audit entire programs or targeted snippets to ensure new or refactored code complies with established application standards.
 - **Tooling integration:** Incorporates rules and checks (including ZCodeScan) directly into Bob's context for automated code reviews.
-

Exercise 6: Architecture Diagram Creation

Objective

Generate a visual program call graph diagram using Mermaid and save it as a Markdown document in the workspace.

Bob Mode to Use

Mode:  **Z Architect**

Z Architect mode specializes in creating architecture diagrams and analyzing application flows.

Context

While the textual inventory provides detailed component listings, a visual call graph makes it easy to understand the control flow, program hierarchy, and transaction routing across the CBSA application at a glance.

Bob Prompt

Generate a program call graph and save it to the workspace.

 **Previewing the Diagram:** The output is saved as a Markdown file containing a Mermaid diagram. Open the generated file and click the **“Open Preview”** icon in the top right of the editor tab to view the rendered visual diagram.

 **Mermaid Rendering Notice:** Generating Mermaid diagrams is not an out-of-the-box native workflow of the IBM Bob Premium Package for Z, so minor syntax errors in the generated diagram can occasionally occur. If the diagram fails to render, simply prompt Bob:

There is an error in your mermaid diagram.

If that is not enough to resolve it, copy and paste the specific error message displayed in the preview into your prompt.

What Bob Does Automatically

Bob queries the local metadata database and scans program linkage information (CICS [LINK](#), [XCTL](#), and transaction routing) to construct the complete hierarchy of calls between presentation modules, service programs, utilities, and batch routines. It then formats this into a Mermaid graph and saves it according to your workspace rules (e.g., [graph/CBSA-archi-program-call-graph.md](#)).

Expected Outcome

Bob generates a Markdown file in the [graph/](#) directory (for example, [graph/CBSA-archi-program-call-graph.md](#)) containing:

- A rendered Mermaid diagram illustrating the program call relationships across the application
- A textual overview of key call paths (presentation programs → controllers → business services → utilities) - ... and more

Key Takeaways

- **Explicit saving instruction:** When asking for diagrams or complex outputs, explicitly prompting Bob to *“save it to the workspace”* ensures the result is persisted to a file rather than only displayed inline in the chat.
- **Metadata-powered visualization:** Bob leverages the metadata database built in earlier exercises along with workspace context to reconstruct accurate call trees across all 28 programs.
- **Rules applied automatically:** Because naming rules were defined in Exercise 2, Bob routes schemas and diagrams directly into the [graph/](#) directory without manual path specification.

Exercise 7: BANKDATA Program Documentation

Objective

Generate role-tailored documentation for the `BANKDATA.cb1` batch initialization program across all three available perspectives (Developer, Business, Architect), then compare how each file targets different stakeholder needs.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

Context

`BANKDATA.cb1` is the batch initialization program responsible for generating initial customer and account test data in VSAM and Db2. Because different roles (developers, enterprise architects, and business analysts) need different information from the same codebase, Bob provides an “**Explain code**” workflow capable of producing documentation customized to each target audience.

Generate the Explanations

Execute the workflow three times to generate documentation for each perspective:

1. Click the **Start Workflow** icon (Play button in Bob).
2. Select “**Explain code**” and click **Start**.
3. Choose `src/base/cobol_src/BANKDATA.cb1` as the program to explain.
4. Select the perspective when prompted:
 - **Run 1:** Select **DEVELOPER**
 - **Run 2:** Repeat the workflow and select **BUSINESS**
 - **Run 3:** Repeat the workflow and select **ARCHITECT**

The workflow automatically saves each generated document to the workspace.

Expected Outcome

Bob automatically creates three distinct Markdown documents in the `docs/explain/` directory:

- `docs/explain/developer_BANKDATA.md` : Technical focus containing low-level paragraph flows, variable descriptions, VSAM/Db2 I/O operations, error codes, and maintenance considerations.
- `docs/explain/business_BANKDATA.md` : Functional focus describing high-level business purposes, customer/account initialization rules, test data generation parameters, and domain logic without technical syntax.
- `docs/explain/architect_BANKDATA.md` : System-level focus covering architectural component roles, integration points with Db2 tables and VSAM datasets, execution prerequisites, and batch design patterns.

 **Compare the Three Perspectives:** Open all three files side by side and observe how Bob adapts the tone, structure, and depth of analysis for each role while examining the exact same underlying COBOL source code.

Key Takeaways

- **Automatic documentation:** Bob analyzes source code and generates structured, comprehensive technical and business documentation in minutes.
- **Target audience context:** Specifying the intended audience or perspective allows Bob to adapt technical depth, terminology, and focus areas to deliver high-value documentation tailored to different stakeholders.
- **Speed and coverage over manual efforts:** Generating complete documentation with diagrams and data dependencies takes minutes instead of days of manual analysis, making it easy to regenerate whenever the code evolves.

 **Where Role-Tailored Documentation Delivers Value:** - **Onboarding & Knowledge Transfer:** Rapidly brings new team members up to speed on unfamiliar legacy modules without pulling senior developers away from active work. - **Maintenance & Modernization:** Provides architects and developers with accurate dependency and parameter mappings before planning code refactoring or migration. - **Audit & Compliance:** Gives business and compliance analysts clear, accessible traceability into initialization rules and data models directly extracted from source.

Exercise 8: Business Rules Extraction and Code Modification

Objective

Walk through a complete developer workflow on a single core program (`BNK1CAC.cbl`): extract business and validation rules into a specification document, implement a new validation rule using in-editor context, resolve a missing copybook diagnostic with Bob Quick Fix, and generate a code enhancement report.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode excels at pattern analysis, code generation, and extracting business rules embedded in COBOL code.

Context

`BNK1CAC.cbl` is the CICS 3270 presentation program responsible for account creation (transaction `OCAC`). It receives customer input from a BMS map, performs validation, calls the backend service `CREACC` , and returns results to the terminal. In this exercise, you will experience how Bob assists across each phase of working with a module: understanding business logic, writing code, resolving diagnostics, and auditing code quality.

Part A: Extract Business Rules

Bob Prompt

```
Extract and save in a md file the business rules from @src/base/cobol_src/BNK1CAC.cbl
```

Expected Outcome

Bob analyzes the program logic and creates a structured specification document in `docs/BNK1CAC/` following the naming rules (e.g. `docs/BNK1CAC/BNK1CAC-spec-business-rules.md`).

While the generated file also contains further technical processing and navigation rules, our primary focus for this exercise is the **Customer Number Validation** section:

- **De-editing:** Customer number input is pre-processed to remove non-numeric formatting characters.
- **Mandatory Presence:** The customer number field must not be blank, un-entered (length `< 1`), or contain mask characters (`' _____ '`).
- **Numeric Integrity:** The customer number must be strictly numeric (`CUSTNOI NOT NUMERIC`).

 **Alternative Workflow:** You can also use the “**Generate program documentation**” workflow (accessible via the Play button) to generate a comprehensive program report that extracts paragraph-by-paragraph business rules alongside full data structures and control flow.

Part B: Implement a New Rule with In-Editor Context

Add a New Business Rule

1. Open `src/base/cobol_src/BNK1CAC.cbl` in the editor.
2. Place your cursor at the beginning of line 458 (press **Ctrl+G** and enter `458`). You should land right below the check verifying that the customer number is numeric.
3. Right-click, and choose **IBM Bob > Add to Context** (this adds `BNK1CAC.cbl:457-459` to Bob’s prompt input).
4. Complete the prompt with instructions to add the validation rule and update the specification document:

BNK1CAC.cbl:457-459 add a test to verify a customer number should start with 99. Also reflect t his change in the rules file you just created.

✓ Expected Outcome

Bob updates both the COBOL source file and the specification document:

1. **BNK1CAC.cbl** : A new validation check is added (starting around line 459) to ensure customer numbers begin with '99' :

```
IF CUSTNOI(1:2) NOT = '99'  
  MOVE SPACES TO MESSAGE0  
  STRING 'Customer number must start with 99'  
    DELIMITED BY SIZE,  
    ' ' DELIMITED BY SIZE  
  INTO MESSAGE0  
  MOVE 'N' TO VALID-DATA-SW  
  MOVE -1 TO CUSTNOL  
  GO TO ED999  
END-IF.
```

💡 **Note on Generated Code:** The exact generated variable names or formatting may vary slightly depending on the context Bob infers, but the core validation logic and error handling flow will be functionally equivalent. (If the changes do not immediately reflect in the open editor tab, simply close and reopen **BNK1CAC.cbl**).

2. **Specification document:** The document is updated to include the new requirement under the customer number validation section along with its associated error message.

Part C: Recreate a Missing Copybook with Bob Quick Fix

🔧 Create Missing Copybook

1. With **BNK1CAC.cbl** open in the editor, open the **Problems** view in the bottom panel. If the panel is not visible, open it via the top application menu bar (**View > Problems**) or press **Ctrl+Shift+M** (Cmd+Shift+M on macOS).
2. Look for the error message indicating: **"Unable to find copybook BNK1CAM"** .
3. Right-click the problem entry and select **Fix with Bob**.

⚠️ **Mode Switch Notice:** Using “Fix with Bob” from the Problems panel automatically switches the active mode from **Z Code** to **Agent** mode. Make sure to manually switch back to **Z Code** mode.

✓ Expected Outcome

Bob analyzes the workspace, identifies that **BNK1CAM** is generated from the corresponding BMS screen map definition, and recreates the missing copybook (**BNK1CAM.cpy**) with the necessary field definitions.

(Note: After **BNK1CAM.cpy** is recreated, remaining diagnostic notices regarding **DFHAID** can be ignored, as **DFHAID** is a standard CICS system copybook provided at runtime).

Part D: Generate a Code Enhancements Report

Bob Prompt

Create a markdown formatted report on all of the possible enhancements that could be made on @src/base/cobol_src/BNK1CAC.cbl

Expected Outcome

Bob conducts a comprehensive code review of `BNK1CAC.cbl` and generates a prioritized enhancement report in `docs/BNK1CAC/` (e.g. `docs/BNK1CAC/BNK1CAC-analysis-enhancements.md`).

The report identifies and categorizes key findings such as: - **Correctness & Bugs:** An incorrect time string construction where `WS-TIME-NOW-GRP-MM` is used twice in the `STRING` statement building `ABND-TIME` (resulting in `HH:MM:MM` instead of `HH:MM:SS`). - **Code Duplication & Refactoring:** Abend handler boilerplate code repeated across five different sections (`A010`, `RM010`, `CAD010`, `SM010`, `STM010`) that can be consolidated into a single routine, saving over 150 lines of duplicate code. - **Complexity & Input Validation:** Overly complex manual digit-by-digit `INSPECT` logic used for interest rate and overdraft validations spanning over 100 lines. - **Control Flow Modernization:** Heavy reliance on unstructured `GO TO` statements for input validation and branch exits that can be modernized with structured `PERFORM` or `EVALUATE` blocks. - ... and more

 **Variation Notice:** Because Bob performs a heuristic analysis on the source code, the exact number of findings, severity ratings, or wording may vary slightly, but it will surface key bugs (like the `ABND-TIME` error), duplications, and modernization opportunities.

Key Takeaways

- **Comprehensive Developer Assistance:** Bob supports the entire developer workflow — from extracting business rules and performing in-editor code modifications, to resolving diagnostic errors via quick fixes and conducting proactive code quality audits.
- **Targeted in-editor modifications:** Using **Add to Context** with specific line ranges allows you to guide Bob in making surgical, localized code edits without unintended side effects on surrounding logic.
- **Context-aware Quick Fix:** Bob uses related assets (such as BMS map definitions) to automatically recreate missing copybooks directly from compiler diagnostics.
- **Proactive Defect & Complexity Discovery:** Automated enhancement reports reveal subtle bugs (e.g., malformed timestamp formatting) and refactoring targets before code reaches production.

Exercise 9: Variable and Data Usage Analysis

Objective

Query Bob about the application to discover and analyze in depth the impact of changing a data element. Here, we start by analyzing a program's variables, then analyze in depth the use of SORTCODE (bank branch code) throughout the application.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode excels at pattern analysis and extracting business rules embedded in COBOL code.

Context

SORTCODE is a critical element: - 6-digit code identifying the bank branch - Used in all tables as a composite key - Current fixed value: 987654

Bob Prompt

What variables are used in INQACCCU?

 **Data Dictionary Selection:** When Bob identifies existing data dictionary sources and asks “How would you like to proceed?”, choose “**Update data dictionary with new variables**” (instead of “Use existing data dictionary as is”). Once Bob presents the updated variable list for review, click “**I’m done editing**” to finalize the updates. If Bob does not prompt you to update the dictionary automatically, you can ask:

Update the existing data dictionary with the variables from INQACCCU.

Expected Outcome

Bob queries the metadata database (`execute_sql_query`) to inventory the variables used in `INQACCCU.cb1` (categorizing COMMAREA interface variables, DB2 host variables `HV-ACCOUNT-*`, internal flags, and database keys like `SORTCODE`).

The new business terms from `INQACCCU` are appended into `bobz/DD.json`, expanding the data dictionary so that it now provides shared semantic definitions across both `BANKDATA` and `INQACCCU`.

Follow-Up Prompt: Investigate Variable Usage

How is the SORTCODE variable used in the application?

Expected Outcome

Bob analyzes references across programs and explains how `SORTCODE` is utilized throughout CBSA: - **Definition:** Originates in `SORTCODE.cpy` as a hardcoded 6-digit constant (`987654`). - **Core service:** Exposed authoritatively via the `GETSCODE` service program. - **Runtime usage patterns:** Used for composite primary key construction in DB2 queries and VSAM browse operations (`INQACC`, `INQACCCU`), input validation (`UPDACC`, `DBCRCFUN`), record stamping upon creation (`CREACC`, `CRECUST`), and CICS named counter scoping. - ... and more

Follow-Up Prompt: Architectural Importance

What is the architectural importance of SORTCODE?

Expected Outcome

Bob synthesizes cross-program evidence and architectural context to explain the broader role of `SORTCODE` : - **Single-tenancy & partitioning**: It acts as the partition key for the entire application, scoping every customer, account, and transaction record to a single branch. - **Cross-cutting reach**: Directly influences the core business logic across inquiry, creation, update, deletion, and transaction programs, as well as persistent DB2 tables (`ACCOUNT` , `PROCTRAN`) and VSAM datasets (`CUSTOMER`). - **Modernization implications**: Serves as a direct precursor to change impact analysis — showing that transitioning from a single hardcoded sort code to a multi-branch architecture requires fundamental data model and program modifications across the entire application.

Key Takeaways

- **Iterative code exploration**: You can have an interactive, exploratory dialogue with Bob to drill down from high-level variable inventories to cross-cutting application patterns without writing complex queries manually.
 - **Data Dictionary enrichment**: Incrementally documenting programs enriches `bobz/DD.json` , building up shared domain terminology across the entire codebase.
 - **Foundation for Change Impact**: Understanding architectural linchpins like `SORTCODE` sets the stage for formal impact analysis and planning in the next exercise.
-

Exercise 10: Change Impact Analysis

Objective

Conduct a comprehensive change impact analysis in **Z Architect** mode to assess the technical scope, effort, risk, and migration requirements of transitioning `SORTCODE` from a single hardcoded branch identifier to a dynamic multi-branch architecture.

Bob Mode to Use

Mode:  **Z Architect**

Z Architect mode specializes in system-level architecture analysis, implementation planning, dependency mapping, and change impact assessment.

 **Switch Mode:** Switch to **Z Architect** mode in Bob before proceeding with this exercise.

Context

In Exercise 9, you discovered that `SORTCODE` serves as the hardcoded single-branch partition key across the entire CBSA application (`SORTCODE.cpy` , `GETSCODE` , DB2 tables, and VSAM datasets). The business now requires modernizing CBSA to support multi-branch operations. Because this modification spans multiple layers—data structures, SQL schemas, CICS screens, and business logic—an architectural impact assessment is essential before making any code changes.

Bob Prompt

 **Keep the Conversation Context:** You can run this prompt in the **same chat session/task** as Exercise 9. Because Bob has already analyzed `SORTCODE` references and data dictionary definitions in the active session, it can directly leverage that conversation context to deliver a more detailed and accurate impact assessment.

Analyze the impact of changing `SORTCODE` to support multiple bank branches.

 **Follow-Up Questions on Architecture Options:** Bob may ask clarifying follow-up questions to understand your target architecture (e.g. data migration strategy, parameter passing method). You can select a specific design option or choose to include all architectural alternatives in the analysis for later evaluation.

Expected Outcome

Bob generates an architecture impact analysis report.

The report provides a comprehensive architectural evaluation: - **Executive Summary & Scope:** Clear classification of in-scope components (all 13 core runtime COBOL programs referencing `SORTCODE` , DB2 tables, VSAM datasets, CICS named counters) vs. out-of-scope presentation pass-through callers. - **Architectural Reach & Dependency Mapping:** Breakdown of how `SORTCODE` permeates data structures (`ACCOUNT` composite key, `CUSTOMER` VSAM key prefix, CICS named counter scoping). - **Migration & Implementation Strategies:** Analysis of candidate approaches (e.g., runtime parameterization vs. environment routing) and data migration scenarios (preserving existing records vs. clean-slate). - **Risk, Effort & Mitigation:** Detailed evaluation of compilation dependencies, database restructuring risks, and suggested testing phases. - ... and more

 **Scope Notice:** As the impact report highlights, transitioning `SORTCODE` to a multi-branch architecture represents a massive, system-wide change affecting 13 core programs, all copybooks, database schemas, and data migration. Due to time constraints, we will not perform this full refactoring during today's lab session, but you are welcome to explore and implement it on your own! In Exercise 12, we will instead implement a more modular feature (Email Search) end-to-end.

Key Takeaways

- **Cross-cutting architectural visibility:** Bob's Z Architect mode analyzes the full stack—from COBOL programs and shared copybooks to DB2 schemas, VSAM files, and CICS middleware—to produce a production-grade impact assessment.
 - **Context preservation:** Running impact analysis in the same task builds directly upon the variable and dependency discoveries made in previous prompts.
 - **Informed decision-making:** The generated impact report maps out architectural options, risks, and migration paths before a single line of code is modified.
-

Exercise 11: User Journey Documentation

Objective

Generate non-technical user journey and operational documentation directly from application flows and BMS map definitions, creating a practical step-by-step guide for bank tellers.

Bob Mode to Use

Mode:  **Z Code**

Z Code mode is specialized for analyzing and documenting mainframe applications (COBOL, PL/I, JCL, Assembler, REXX).

 **Switch Mode:** Switch back to **Z Code** mode in Bob before running this prompt.

Context

Beyond producing technical specifications and architecture assessments, Bob can translate low-level screen flows, navigation transactions, and business validation logic into user-facing operational guides. In this exercise, you will instruct Bob to trace the account inquiry path starting from the main menu (`BNKMENU` / transaction `OMEN`) through the consultation screen (`BNK1CCA` / transaction `OCCA`) and backend service (`INQACCCU`), formatting the result as an end-user guide for bank tellers.

Bob Prompt

Create user journey documentation for consulting a customer's accounts in the CBSA application.

The documentation must:

- Be intended for tellers (non-technical)
- Show the path from the main menu
- Include screen examples
- Provide practical tips

Expected Outcome

Bob generates an end-user operational document in the `docs/` directory respecting the naming rules defined in Exercise 2 (for example, `docs/CBSA-docu-teller-account-inquiry-journey.md`).

The generated document includes:

- **Multi-Path Consultation Overview:** Clear breakdown of account inquiry routes depending on available customer data (e.g., lookup by Customer Number, full profile inquiry, or direct account lookup).
- **Simulated 3270 Terminal Screens:** Accurate ASCII visual layouts of the Main Menu (`BNK1MA`) and Accounts for Customers screen (`BNK1ACC`) directly derived from BMS map definitions.
- **Teller Step-by-Step Guidance:** Clear instructions on entering 10-digit customer identifiers, navigating screen fields, reading account lists (sort codes, account types, available vs. actual balances), and using function keys (`F3=Exit` , `F12=Cancel`).
- **Error Messages & Troubleshooting:** Practical explanations for common teller scenarios, including invalid customer numbers, empty account lists, and error message area interpretations.
- ... and more

Key Takeaways

- **Bridging Technical Code to Business Documentation:** Bob reads BMS screen maps, program control flows, and COMMAREA definitions to generate user-friendly operational manuals without requiring manual translation from technical specs.
- **Simulated Terminal Views:** Screen mockups and keystroke navigation paths are automatically derived from source assets, reducing the need for manual screenshot gathering.

- **Adaptability across Stakeholder Needs:** Complements technical and architectural outputs by delivering business-level documentation for operational and front-office teams.
-

Exercise 12: Email Search Implementation

Objective

Design, plan, and implement an end-to-end new feature in the CBSA application: enable customer lookup by email address. You will transition from architectural planning to data structure updates, COBOL program generation, and syntax verification.

Context

The business wants to modernize the CBSA banking application by allowing customers to be identified by their email address rather than exclusively relying on a 10-digit customer number.

To implement this feature safely and maintain system compatibility, the process follows a structured four-part development lifecycle: - **Part A (Z Architect)**: Architecture & Implementation Planning - **Part B (Z Code)**: Data Structure Modification - **Part C (Z Code)**: Search Program Development - **Part D (Z Code)**: Program Syntax Verification

Before creating the plan, you can query Bob in your current **Z Code** mode to confirm the application's existing customer search capabilities:

Optional Exploratory Prompt

What criteria can be used to search for a customer in CBSA?

Bob will explain that customer inquiries currently rely on Customer Number only, confirming the motivation for adding an alternative. We will do this by implementing an email-based search.

Part A: Implementation Planning

Bob Mode to Use

Mode:  **Z Architect**

Z Architect mode is designed for system-level planning, breaking down architectural changes, defining component impacts, and structuring multi-phase implementation plans.

 **Switch Mode:** Switch to **Z Architect** mode in Bob before running this prompt.

Bob Prompt

Create a detailed implementation plan to enable customer search by email address using a new dedicated subprogram named INQEMAIL. Save the plan to the workspace.

 **Follow-Up Design Questions:** Bob will ask clarifying questions regarding your architectural preferences (e.g. storage approach, parameterization). You can discuss these choices interactively with Bob — for instance, by asking “Which option would you recommend for the best performance?” — or make a selection based on your preference. Note that depending on your choices, the resulting architecture may slightly differ from the Expected Outcomes in the following parts.

Expected Outcome

Bob analyzes the application architecture and generates a structured implementation plan.

The generated plan outlines: - **Executive summary & business objectives:** Functional goal of adding email-based customer lookup alongside existing search modes - **Architecture & component design:** Specification of the new [INQEMAIL.cb1](#) CICS

search subprogram and its COMMAREA interface ([INQEMAIL.cpy](#)) - **Data model modifications:** Required updates to customer record copybooks ([CUSTOMER.cpy](#)) and transaction COMMAREAs to accommodate email fields and verification flags - **Implementation roadmap:** Step-by-step phased approach covering data structures, subprogram development, VSAM alternate index considerations, and test strategies - ... and more

🔑 Key Takeaways

- **Explicit naming in prompts:** Specifying desired program names like [INQEMAIL](#) directly in your prompt ensures Bob aligns all generated artifacts, plans, and copybooks consistently with your project conventions rather than choosing auto-generated names.
- **Architectural roadmap before coding:** Generating a formal plan in Z Architect mode maps out every dependent structure and interface before modifying any COBOL files.

Part B: Data Structure Modification

🔧 Bob Mode to Use

Mode: 📦 **Z Code**

Z Code mode specializes in parsing, modifying, and generating mainframe copybooks, data structures, and COBOL programs.

⚠️ **Switch Mode:** Switch to **Z Code** mode in Bob before running this prompt.

💬 Bob Prompt

According to the implementation plan, update the CUSTOMER data structures.

✅ Expected Outcome

Bob executes the data model changes outlined in your implementation plan by modifying existing copybooks and generating any required new copybooks in [src/base/cobol_copy/](#) (e.g. [CUSTOMER.cpy](#) , [INQCUST.cpy](#) , [CRECUST.cpy](#) , [UPDCUST.cpy](#) , [INQEMAIL.cpy](#) , or [CUSTEML.cpy](#)).

Typical updates include: - **Customer Record Layout ([CUSTOMER.cpy](#)) / Table Definitions:** Adding an email field definition (and optional verification status flags), while preserving layout integrity or adjusting filler bytes. - **Interface COMMAREAs ([INQCUST.cpy](#) , [CRECUST.cpy](#) , [UPDCUST.cpy](#)):** Extending customer service communication areas with email input/output fields. - **New Search COMMAREA ([INQEMAIL.cpy](#)):** Defining the communication contract for the new [INQEMAIL](#) subprogram (e.g., input email query field, returned customer identifier, response codes, and status messages).

💡 **Design Variation Notice:** The exact copybooks modified, variable names, and field lengths (e.g. 80 vs. 100 characters) will directly reflect the design choices made in your implementation plan from Part A.

💡 **Skill Invocation in Action:** Notice that Bob automatically loads and adheres to the **CBSA coding standards skill** created in Exercise 5 ([.bob/skills/cbsa-coding-standards/SKILL.md](#)) when formatting variable declarations, prefix conventions, and copybook layouts.

🔑 Key Takeaways

- **Holistic data contract updates:** Bob coordinates modifications across all related copybooks simultaneously to ensure interface consistency between caller programs and backend services.
- **Design consistency & Standards adherence:** Data structures are updated in alignment with the architecture decisions documented in the implementation plan while automatically respecting the coding standards skill created earlier.

Part C: Search Program Development

Bob Mode to Use

Mode:  Z Code

Z Code mode specializes in generating clean, structured COBOL code and CICS command syntax for z/OS.

Bob Prompt

Create the INQEMAIL program according to the implementation plan

Expected Outcome

Bob generates the new CICS search program at `src/base/cobol_src/INQEMAIL.cbl` implementing the functionality specified in the implementation plan: - **Interface & Linkage:** Includes `COPY INQEMAIL` for its input/output COMMAREA interface and references the relevant customer data structures. - **Search Logic:** Implements the lookup routine according to your architecture choices (e.g. querying a new `CUSTOMER_EMAIL` DB2 table with embedded SQL, or searching via VSAM index, then linking to `INQCUST`). - **Input Validation:** Verifies email formatting, length, and mandatory presence before initiating datastore lookups. - **CICS Return Code & Error Handling:** Evaluates CICS response codes (`DFHRESP`), returns standardized response codes/messages via the COMMAREA, and routes unexpected system failures to the `ABNDPROC` error handler.

 **Behind the Scenes Observations:** - **Design Alignment:** The exact SQL queries, VSAM commands, and validation routines in `INQEMAIL.cbl` directly reflect the architecture selected in your plan from Part A. - **Context Efficiency:** With an implementation plan and workspace rules in place, prompts can remain concise without needing to repeat low-level field positions or copybook lists. - **Standards & Automated Review:** Bob automatically applies the previously loaded CBSA coding standards skill and spawns a background code review subagent to inspect the generated COBOL for syntax, alignment, and compliance before presenting the result.

Key Takeaways

- **Production-ready COBOL generation:** Bob writes fully structured, commented CICS COBOL code with complete error handling, `EXEC CICS` commands, and response code evaluations.
- **Strict adherence to project standards:** Variable naming, paragraph layout, and abend handling routines conform to the CBSA coding standards established earlier.

Part D: Program Syntax Verification

Bob Mode to Use

Mode:  Z Code

Z Code mode specializes in code validation, parsing division structures, verifying copybook linkages, and checking CICS/SQL syntax.

Bob Prompt

Verify the syntax of `@src/base/cobol_src/INQEMAIL.cbl`

Expected Outcome

Bob performs a comprehensive static syntax and structural verification of `INQEMAIL.cbl`, reporting on: - **COBOL Division Structure:** Verifies that `IDENTIFICATION`, `ENVIRONMENT`, `DATA`, and `PROCEDURE` divisions are properly declared and ordered. - **Copybook & Linkage Resolution:** Confirms that copybook references (e.g. `COPY INQEMAIL`, `COPY ABNDINFO`,

`COPY CUSTOMER / CUSTEML`) resolve cleanly in `src/base/cobol_copy/` . - **Variable Declarations & Types:** Checks that all referenced variables and host fields are declared with consistent `PIC` types and initialized where necessary. - **CICS & SQL Syntax:** Validates `EXEC CICS` commands (such as `READ` , `LINK` , `RETURN`) and embedded SQL statements, confirming proper condition and return code handling. - **Standards & Best Practices:** Checks adherence to CBSA coding conventions, structured control flow, and clean termination paths.

Key Takeaways

- **Pre-compilation quality gate:** Static verification within Bob catches missing copybooks, undeclared variables, or malformed CICS/SQL statements before submitting jobs to z/OS.
 - **Full lifecycle coverage:** Exercise 12 demonstrates how Bob guides developers through the entire lifecycle of a new feature — from architecture planning to data model updates, code generation, and syntax verification.
-

Summary and Measurable Gains

Lab Journey & Efficiency Gains

Throughout this hands-on lab, you applied IBM Bob Premium Package for Z across the entire lifecycle of a mainframe modernization initiative — from initial repository discovery and rule definition to architectural planning, deep static analysis, business rule extraction, and end-to-end feature development.

Exercise	Focus Area	Primary Mode	Key Deliverable	Manual Effort	With Bob	Efficiency Gain
Ex 0–2	Workspace Setup & Governance	 Agent /  Z Code	<code>AGENTS.md</code> , <code>.bob/rules/cbsa-workspace-rules.md</code>	1–2 days	~5 min	~99%
Ex 3–4	Metadata & Inventory	 Z Code	<code>bobz/DD.json</code> , <code>docs/CBSA-inv-application-inventory.md</code>	1–2 weeks	~10 min	~98%
Ex 5	Reusable Standards Skill	 Z Code	<code>.bob/skills/cbsa-coding-standards/SKILL.md</code>	2–3 days	~5 min	~99%
Ex 6	Architecture Visualisation	 Z Architect	<code>graph/CBSA-archi-program-call-graph.md</code>	2–3 days	~5 min	~99%
Ex 7	Multi-Perspective Docs	 Z Code	<code>docs/explain/</code> (3 perspectives)	2–3 days	~5 min	~99%
Ex 8	Rule Extraction & Refactoring	 Z Code	<code>docs/BNK1CAC/</code> , <code>src/base/cobol_src/BNK1CAC.cb1</code>	3–4 days	~15 min	~95%
Ex 9–10	Usage & Multi-Branch Impact	 Z Code /  Z Architect	<code>docs/SORTCODE/</code> , <code>docs/CBSA-impact-sortcode-analysis.md</code>	1–2 weeks	~20 min	~98%
Ex 11	User Journey Documentation	 Z Code	<code>docs/CBSA-docu-teller-account-inquiry-journey.md</code>	2–3 days	~10 min	~98%
Ex 12	End-to-End Implementation	 Z Architect /  Z Code	<code>docs/INQEMAIL/</code> , <code>INQEMAIL.cpy</code> , <code>INQEMAIL.cb1</code>	2–3 weeks	~45 min	~95%
Total	Full Application Lifecycle	—	Complete, standard-compliant assets	8–12 weeks	~2 hours	~98%

High-Level Added Value of IBM Bob for Z

- Intelligent Legacy Comprehension:** Instantly maps cross-program CICS and batch call chains, resolves copybook linkages, and generates interactive Mermaid diagrams without manual tracing.
- Deterministic Governance & Quality:** Enforces workspace rules, project naming conventions, and custom enterprise coding standards autonomously across all generated code and documentation.

3. **Multi-Perspective Knowledge Capture:** Simultaneously produces Architect, Business, and Developer documentation from raw mainframe source code, bridging the gap between technical teams and business stakeholders.
 4. **Confident Change Management:** Performs deep multi-branch impact analyses and data dictionary tracking ([bobz/DD.json](#)) before code changes occur, preventing runtime regressions in critical transaction paths.
 5. **Accelerated Feature Delivery:** Converts high-level architectural requirements into complete, standard-compliant CICS COBOL copybooks, data structures, and programs with automated pre-compilation syntax validation.
-

Conclusion

Congratulations!

You have completed the **IBM Bob Premium Package for Z** lab. You are now equipped to navigate complex mainframe workspaces, automate architectural and business documentation, perform rigorous impact analyses, and accelerate software development on z/OS using AI assistance.

 **Note:** The sections below provide general reference material and recommended next steps beyond this lab. They are not part of the hands-on lab exercises.

Appendix: Useful Prompt Reference (Optional Reference)

Use these prompt patterns as templates when applying Bob to your own z/OS projects:

Code Analysis & Understanding

```
Explain what the @[PROGRAM].cbl program does
What are the dependencies of the @[PROGRAM].cbl program?
Where is the [VARIABLE] variable used across the workspace?
What copybooks are referenced by @[PROGRAM].cbl?
```

Impact & Data Flow Analysis

```
What is the impact of modifying [FIELD] in @[COPYBOOK].cpy?
Which programs are affected if we expand [DATA_STRUCTURE]?
What are the business rules associated with [DATA_ELEMENT]?
Analyze the call chain leading to @[PROGRAM].cbl
```

Documentation & Architecture

```
Create operational documentation for the [FEATURE/TRANSACTION] flow
Document the architecture and data store interactions of [MODULE]
Generate a comprehensive application inventory of all COBOL assets
Create a Mermaid program call graph for the [SYSTEM] component
```

Assisted Development & Refactoring

```
Create a new CICS COBOL program for [FEATURE] adhering to our workspace rules
Modify @[PROGRAM].cbl to add validation for [REQUIREMENT]
Generate the COMMAREA copybook for [NEW_SERVICE]
Verify the static syntax and division structure of @[PROGRAM].cbl
```

Next Steps: Beyond the Lab (Reference Guide)

Applying Bob in Your Organization

1. **Bootstrap Your Applications:** Run `/init` and execute metadata scans on your existing mainframe repositories to build foundational data dictionaries and workspace profiles.
2. **Encode Enterprise Standards:** Create team-specific rules in `.bob/rules/` and reusable skills in `.bob/skills/` to standardize naming conventions, error-handling routines, and coding guidelines.
3. **Streamline Day-to-Day Maintenance:** Integrate Z Architect for pre-change impact analyses and Z Code for rapid bug fixes, documentation generation, and feature enhancements.
4. **Knowledge Democratization:** Use multi-perspective code explanations to accelerate onboarding for junior developers and facilitate communication between technical leads and business analysts.

Additional Resources

- **Bob Product Documentation:** Consult official user guides for advanced tool usage, mode configurations, and custom skill building.
- **Support & Feedback:** Connect with the IBM Bob Premium Package for Z engineering and support teams.
- **Community:** Join the IBM Z user community to share custom skills, workflow patterns, and best practices.

Version: 2.0

Author: IBM Bob Premium Package for Z Team

Transform the way you work with mainframe applications! 